

CSS Transform 3D Properties

Complete Guide: From Basics to Advanced

Contents

1	Introduction	2
1.1	What are 3D Transforms?	2
1.2	2D vs. 3D Transforms	2
1.3	The 3D Coordinate System	2
1.4	Why Use 3D Transforms?	2
2	Perspective: Creating Depth	3
2.1	perspective Property	3
2.1.1	Syntax	3
2.1.2	Parameters	3
2.1.3	Examples	3
2.2	perspective-origin	4
2.2.1	Syntax	4
2.2.2	Values	4
2.2.3	Examples	4
2.3	transform-style: preserve-3d	5
2.3.1	Syntax	5
2.3.2	Critical for 3D Composition	5
3	3D Rotation Functions	7
3.1	rotateX()	7
3.1.1	Syntax	7
3.1.2	Examples	7
3.2	rotateY()	8
3.2.1	Syntax	8
3.2.2	Examples	8
3.3	rotateZ()	8
3.3.1	Syntax	9
3.3.2	Examples	9
3.4	rotate3d()	9
3.4.1	Syntax	9
3.4.2	Parameters	9
3.4.3	Examples	9
4	3D Translation and Scaling	11
4.1	translateZ()	11
4.1.1	Syntax	11

4.1.2	Parameters	11
4.1.3	Examples	11
4.2	translate3d()	12
4.2.1	Syntax	12
4.2.2	Examples	12
4.3	scale3d()	12
4.3.1	Syntax	12
4.3.2	Examples	13
5	Combining 3D Transforms	14
5.1	Using Multiple 3D Transforms	14
5.1.1	Syntax	14
5.2	Complex 3D Effects	14
6	Backface Visibility	16
6.1	backface-visibility Property	16
6.1.1	Syntax	16
6.1.2	Values	16
6.2	Use Cases	16
7	Real-World Use Cases	18
7.1	Card Flip on Click	18
7.2	3D Carousel	19
7.3	Floating 3D Box	20
7.4	Product Showcase with 3D Rotation	21
7.5	Interactive 3D Photo Gallery	22
8	Advanced Techniques	23
8.1	Matrix3d	23
8.1.1	Syntax	23
8.1.2	Note	23
8.2	Combining with JavaScript for Interactivity	23
8.3	Parallax 3D Effect	24
8.4	Performance Optimization	24
9	Browser Support & Best Practices	26
9.1	Browser Support	26
9.1.1	Vendor Prefixes	26
9.2	Performance Best Practices	26
9.3	Common Mistakes	27
9.3.1	Mistake 1: Forgetting perspective	27
9.3.2	Mistake 2: Not Using transform-style: preserve-3d	27
9.3.3	Mistake 3: Incorrect backface-visibility	27
9.4	Tips for Better 3D Effects	27

10 Quick Reference Guide	28
10.1 3D Transform Functions	28
10.2 3D Properties	28
10.3 Common Combinations	28
10.3.1 Card Flip	28
10.3.2 3D Rotate and Float	29
10.3.3 Cube	29
10.3.4 Spinning Element	29
10.4 Do's and Don'ts	29
10.5 Perspective Value Guide	29
10.6 CSS 3D Transform Checklist	30

Chapter 1

Introduction

1.1 What are 3D Transforms?

CSS 3D transforms allow you to rotate, scale, and position HTML elements in three-dimensional space. This creates depth effects, flip animations, and immersive visual experiences all with pure CSS!

Think of 3D transforms like turning a flat piece of paper into a box you can rotate in space!

1.2 2D vs. 3D Transforms

- **2D Transforms:** Rotate, scale, move on a flat plane (X, Y axes only)
- **3D Transforms:** Add depth dimension (X, Y, Z axes) for realistic 3D effects
- **2D use case:** Basic animations, hover effects, simple movements
- **3D use case:** Flip cards, rotating cubes, depth effects, immersive UI

1.3 The 3D Coordinate System

- **X-axis:** Horizontal (left-right)
- **Y-axis:** Vertical (up-down)
- **Z-axis:** Depth (toward/away from viewer)

1.4 Why Use 3D Transforms?

- **Depth Perception:** Makes UI feel more modern and interactive
- **Engaging Effects:** Flip cards, rotating cubes, layered animations
- **GPU Accelerated:** Smooth 60fps performance
- **No Images Needed:** Create 3D effects with pure CSS
- **Interactive:** Respond to user interactions with depth

Chapter 2

Perspective: Creating Depth

2.1 perspective Property

Creates a 3D space for child elements. Without perspective, 3D transforms won't have depth effect.

2.1.1 Syntax

```
perspective: value;
```

2.1.2 Parameters

- Value in pixels (px)
- Lower values: More dramatic 3D effect (100px = extreme)
- Higher values: Subtle 3D effect (1000px = subtle)
- Common range: 800px - 1200px

2.1.3 Examples

Subtle Perspective (1000px)

```
.container {  
  perspective: 1000px;  
}  
  
.card {  
  transform: rotateY(45deg);  
}
```

Dramatic Perspective (500px)

```
.container {  
  perspective: 500px;  
}
```

```
.cube {  
  transform: rotateX(45deg) rotateY(45deg);  
}
```

Much more extreme 3D effect.

Extreme Perspective (200px)

```
.container {  
  perspective: 200px;  
}  
  
.element {  
  transform: rotateY(60deg);  
}
```

Very dramatic, exaggerated perspective.

2.2 perspective-origin

Sets the vanishing point for perspective (where lines converge).

2.2.1 Syntax

```
perspective-origin: x y;
```

2.2.2 Values

- Keywords: left, center, right, top, bottom
- Percentages: 0%, 50%, 100%
- Pixels: 10px, 100px, etc.
- Default: center center (50% 50%)

2.2.3 Examples

Vanishing Point at Top-Left

```
.container {  
  perspective: 1000px;  
  perspective-origin: top left;  
}
```

Vanishing Point at Bottom-Right

```
.container {  
  perspective: 800px;  
  perspective-origin: bottom right;  
}
```

2.3 transform-style: preserve-3d

Allows child elements to maintain their 3D positioning.

2.3.1 Syntax

```
transform-style: preserve-3d;
```

2.3.2 Critical for 3D Composition

Creating a 3D Cube

```
.cube {  
  perspective: 1000px;  
}  
  
.cube-faces {  
  transform-style: preserve-3d;  
  transform: rotateX(45deg) rotateY(45deg);  
  position: relative;  
  width: 200px;  
  height: 200px;  
}  
  
.cube-faces .face {  
  position: absolute;  
  width: 100%;  
  height: 100%;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  opacity: 0.8;  
}  
  
.cube-faces .front {  
  background: red;  
  transform: translateZ(100px);  
}  
  
.cube-faces .back {  
  background: blue;  
  transform: rotateY(180deg) translateZ(100px);  
}
```

```
.cube-faces .right {  
  background: green;  
  transform: rotateY(90deg) translateZ(100px);  
}  
.cube-faces .left {  
  background: yellow;  
  transform: rotateY(-90deg) translateZ(100px);  
}  
.cube-faces .top {  
  background: purple;  
  transform: rotateX(90deg) translateZ(100px);  
}  
.cube-faces .bottom {  
  background: orange;  
  transform: rotateX(-90deg) translateZ(100px);  
}
```

Chapter 3

3D Rotation Functions

3.1 rotateX()

Rotates element around the X-axis (tilts forward/backward).

3.1.1 Syntax

```
transform: rotateX(angle);
```

3.1.2 Examples

Tilt Forward

```
.card {  
  perspective: 1000px;  
}  
  
.card-inner {  
  transform: rotateX(45deg);  
}
```

Tilts the card forward (top edge closer to viewer).

Tilt Backward

```
.element {  
  transform: rotateX(-45deg);  
}
```

Page Flip Animation

```
.page {  
  animation: flip-page 1s ease-in-out;  
}  
  
@keyframes flip-page {
```

```
0% { transform: rotateX(0deg); }  
50% { transform: rotateX(90deg); }  
100% { transform: rotateX(180deg); }  
}
```

3.2 rotateY()

Rotates element around the Y-axis (spins left/right).

3.2.1 Syntax

```
transform: rotateY(angle);
```

3.2.2 Examples

Spin Right

```
.card {  
  perspective: 1000px;  
  transform: rotateY(45deg);  
}
```

Spin Left

```
.element {  
  transform: rotateY(-45deg);  
}
```

Card Flip Effect

```
.card {  
  perspective: 1000px;  
  transition: transform 0.6s;  
}  
  
.card:hover {  
  transform: rotateY(180deg);  
}
```

Flips card horizontally on hover.

3.3 rotateZ()

Rotates element around the Z-axis (spins in place, like 2D rotate).

3.3.1 Syntax

```
transform: rotateZ(angle);
```

3.3.2 Examples

Spin in Place

```
.element {  
  transform: rotateZ(45deg);  
}
```

Spinning Icon

```
.icon {  
  animation: spin 2s linear infinite;  
}  
  
@keyframes spin {  
  from { transform: rotateZ(0deg); }  
  to { transform: rotateZ(360deg); }  
}
```

3.4 rotate3d()

Creates custom 3D rotation along any axis.

3.4.1 Syntax

```
transform: rotate3d(x, y, z, angle);
```

3.4.2 Parameters

- x, y, z: Direction vector (0-1)
- angle: Rotation amount in degrees

3.4.3 Examples

Rotate Around Diagonal Axis

```
.element {  
  transform: rotate3d(1, 1, 0, 45deg);  
}
```

Complex Rotation

```
.cube {  
  transform: rotate3d(1, 1, 1, 60deg);  
}
```

Chapter 4

3D Translation and Scaling

4.1 translateZ()

Moves element forward/backward in 3D space (toward/away from viewer).

4.1.1 Syntax

```
transform: translateZ(value);
```

4.1.2 Parameters

- Positive values: Toward viewer (moves forward)
- Negative values: Away from viewer (moves backward)
- Units: px only (no percentages)

4.1.3 Examples

Bring Forward

```
.box {  
  perspective: 1000px;  
  transform: translateZ(100px);  
}
```

Push Back

```
.element {  
  transform: translateZ(-50px);  
}
```

Floating Effect

```
.card {  
  perspective: 1000px;  
  transition: transform 0.3s;  
}  
  
.card:hover {  
  transform: translateZ(50px);  
}
```

4.2 translate3d()

Combines X, Y, and Z translation.

4.2.1 Syntax

```
transform: translate3d(x, y, z);
```

4.2.2 Examples

Move in All Directions

```
.element {  
  transform: translate3d(30px, 50px, 100px);  
}
```

Diagonal Approach

```
.box {  
  animation: approach 2s ease-in-out;  
}  
  
@keyframes approach {  
  from { transform: translate3d(-100px, -100px, -200px); }  
  to { transform: translate3d(0, 0, 0); }  
}
```

4.3 scale3d()

Scales element in 3D space.

4.3.1 Syntax

```
transform: scale3d(x, y, z);
```

4.3.2 Examples

Uniform 3D Scale

```
.element {  
  transform: scale3d(1.5, 1.5, 1.5);  
}
```

Non-uniform Scaling

```
.element {  
  transform: scale3d(2, 1, 1.5);  
}
```

Hover Zoom with Depth

```
.card {  
  perspective: 1000px;  
  transition: transform 0.3s;  
}  
  
.card:hover {  
  transform: scale3d(1.1, 1.1, 1.1) translateZ(20px);  
}
```

Chapter 5

Combining 3D Transforms

5.1 Using Multiple 3D Transforms

You can combine multiple 3D transforms on a single element!

5.1.1 Syntax

```
transform: rotateX(45deg) rotateY(30deg) translateZ(100px);
```

5.2 Complex 3D Effects

Flip and Float

```
.card {  
  perspective: 1000px;  
  transition: transform 0.6s ease;  
}  
  
.card:hover {  
  transform: rotateY(180deg) translateZ(50px);  
}
```

3D Cube Rotation

```
.cube {  
  perspective: 1000px;  
  animation: rotate-cube 10s infinite linear;  
}  
  
@keyframes rotate-cube {  
  0% { transform: rotateX(0deg) rotateY(0deg); }  
  100% { transform: rotateX(360deg) rotateY(360deg); }  
}
```

Tilt and Lift

```
.element {
  perspective: 1000px;
  transition: transform 0.3s ease;
}

.element:hover {
  transform: rotateX(-10deg) rotateY(10deg)
            translateZ(30px) scale3d(1.05, 1.05, 1.05);
}
```

Spiral Animation

```
.element {
  animation: spiral 3s ease-in-out infinite;
}

@keyframes spiral {
  0% {
    transform: rotateX(0deg) rotateY(0deg)
              rotateZ(0deg) translateZ(0);
  }
  50% {
    transform: rotateX(180deg) rotateY(180deg)
              rotateZ(180deg) translateZ(100px);
  }
  100% {
    transform: rotateX(360deg) rotateY(360deg)
              rotateZ(360deg) translateZ(0);
  }
}
```

Tumbling Box

```
.box {
  animation: tumble 2s ease-in-out infinite;
}

@keyframes tumble {
  0% { transform: rotateX(0) rotateY(0) rotateZ(0); }
  25% { transform: rotateX(90deg) rotateY(45deg); }
  50% { transform: rotateX(180deg) rotateY(90deg); }
  75% { transform: rotateX(270deg) rotateY(135deg); }
  100% { transform: rotateX(360deg) rotateY(180deg); }
}
```

Chapter 6

Backface Visibility

6.1 backface-visibility Property

Controls whether an element's back side is visible when rotated away from viewer.

6.1.1 Syntax

```
backface-visibility: visible | hidden;
```

6.1.2 Values

- **visible**: Shows back face (default)
- **hidden**: Hides back face (only shows front)

—

6.2 Use Cases

Card Flip Effect

```
<style>
  .card-container {
    perspective: 1000px;
    width: 200px;
    height: 250px;
  }

  .card-inner {
    position: relative;
    width: 100%;
    height: 100%;
    transition: transform 0.6s;
    transform-style: preserve-3d;
  }

  .card-container:hover .card-inner {
```

```

    transform: rotateY(180deg);
  }

  .card-face {
    position: absolute;
    width: 100%;
    height: 100%;
    backface-visibility: hidden;
    display: flex;
    align-items: center;
    justify-content: center;
  }

  .card-front {
    background: blue;
    color: white;
  }

  .card-back {
    background: red;
    color: white;
    transform: rotateY(180deg);
  }
</style>

<div class="card-container">
  <div class="card-inner">
    <div class="card-face card-front">
      <h3>Front</h3>
    </div>
    <div class="card-face card-back">
      <h3>Back</h3>
    </div>
  </div>
</div>

```

Rotating Cube with Hidden Backs

```

.cube-face {
  backface-visibility: hidden;
  position: absolute;
  width: 200px;
  height: 200px;
}

.cube-front { transform: translateZ(100px); }
.cube-back { transform: rotateY(180deg) translateZ(100px); }
.cube-right { transform: rotateY(90deg) translateZ(100px); }
.cube-left { transform: rotateY(-90deg) translateZ(100px); }
.cube-top { transform: rotateX(90deg) translateZ(100px); }
.cube-bottom { transform: rotateX(-90deg) translateZ(100px); }

```

Chapter 7

Real-World Use Cases

7.1 Card Flip on Click

```
<style>
.flip-card {
  width: 300px;
  height: 400px;
  perspective: 1000px;
  cursor: pointer;
}

.flip-card-inner {
  position: relative;
  width: 100%;
  height: 100%;
  text-align: center;
  transition: transform 0.6s;
  transform-style: preserve-3d;
}

.flip-card.flipped .flip-card-inner {
  transform: rotateY(180deg);
}

.flip-card-front, .flip-card-back {
  position: absolute;
  width: 100%;
  height: 100%;
  backface-visibility: hidden;
  display: flex;
  align-items: center;
  justify-content: center;
}

.flip-card-front {
  background: linear-gradient(135deg, #667eea, #764ba2);
  color: white;
}
```

```

.flip-card-back {
  background: #f093fb;
  color: white;
  transform: rotateY(180deg);
}
</style>

<div class="flip-card">
  <div class="flip-card-inner">
    <div class="flip-card-front">
      <h1>Front</h1>
    </div>
    <div class="flip-card-back">
      <h1>Back</h1>
    </div>
  </div>
</div>

<script>
  const card = document.querySelector('.flip-card');
  card.addEventListener('click', function() {
    this.classList.toggle('flipped');
  });
</script>

```

7.2 3D Carousel

```

<style>
  .carousel {
    width: 300px;
    height: 300px;
    perspective: 1000px;
    position: relative;
  }

  .carousel-inner {
    position: relative;
    width: 100%;
    height: 100%;
    transform-style: preserve-3d;
    animation: rotate-carousel 20s infinite linear;
  }

  .carousel-item {
    position: absolute;
    width: 100%;
    height: 100%;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 48px;
  }

```

```

}

.carousel-item:nth-child(1) {
  transform: rotateY(0deg) translateZ(300px);
  background: #ff6b6b;
}
.carousel-item:nth-child(2) {
  transform: rotateY(120deg) translateZ(300px);
  background: #4ecdc4;
}
.carousel-item:nth-child(3) {
  transform: rotateY(240deg) translateZ(300px);
  background: #45b7d1;
}

@keyframes rotate-carousel {
  from { transform: rotateY(0deg); }
  to { transform: rotateY(360deg); }
}
</style>

<div class="carousel">
  <div class="carousel-inner">
    <div class="carousel-item">1</div>
    <div class="carousel-item">2</div>
    <div class="carousel-item">3</div>
  </div>
</div>

```

7.3 Floating 3D Box

```

<style>
  .floating-box {
    width: 200px;
    height: 200px;
    animation: float-3d 3s ease-in-out infinite;
    perspective: 1000px;
  }

  .floating-box-inner {
    width: 100%;
    height: 100%;
    background: linear-gradient(135deg, #667eea, #764ba2);
    border-radius: 10px;
    box-shadow: 0 10px 40px rgba(0, 0, 0, 0.3);
    transform-style: preserve-3d;
  }

  @keyframes float-3d {
    0% {
      transform: translateY(0) rotateX(0deg) rotateY(0deg);
    }
  }

```

```

    }
    50% {
        transform: translateY(-20px) rotateX(10deg) rotateY(10deg);
    }
    100% {
        transform: translateY(0) rotateX(0deg) rotateY(0deg);
    }
}
</style>

<div class="floating-box">
  <div class="floating-box-inner"></div>
</div>

```

7.4 Product Showcase with 3D Rotation

```

<style>
.product-viewer {
    width: 400px;
    height: 400px;
    perspective: 1000px;
    background: white;
    border-radius: 10px;
    overflow: hidden;
}

.product-model {
    width: 100%;
    height: 100%;
    background-image: url('product.jpg');
    background-size: contain;
    background-repeat: no-repeat;
    background-position: center;
    animation: rotate-product 6s linear infinite;
}

@keyframes rotate-product {
    from { transform: rotateY(0deg); }
    to { transform: rotateY(360deg); }
}

.product-viewer:hover .product-model {
    animation-play-state: paused;
}
</style>

<div class="product-viewer">
  <div class="product-model"></div>
</div>

```

7.5 Interactive 3D Photo Gallery

```
<style>
.gallery-3d {
  perspective: 1000px;
  width: 600px;
  height: 400px;
}

.gallery-3d-inner {
  width: 100%;
  height: 100%;
  transform-style: preserve-3d;
  position: relative;
}

.gallery-3d-item {
  position: absolute;
  width: 300px;
  height: 300px;
  left: 150px;
  top: 50px;
  backface-visibility: hidden;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);
}

.gallery-3d-item:nth-child(1) {
  transform: rotateY(0deg) translateZ(100px);
}

.gallery-3d-item:nth-child(2) {
  transform: rotateY(90deg) translateZ(100px);
}

.gallery-3d-item:nth-child(3) {
  transform: rotateY(180deg) translateZ(100px);
}

.gallery-3d-item:nth-child(4) {
  transform: rotateY(270deg) translateZ(100px);
}
</style>

<div class="gallery-3d">
  <div class="gallery-3d-inner">
    
    
    
    
  </div>
</div>
```

Chapter 8

Advanced Techniques

8.1 Matrix3d

Advanced function for complex 3D transformations using a 4x4 matrix.

8.1.1 Syntax

```
transform: matrix3d(n,n,n,n, n,n,n,n,  
                   n,n,n,n, n,n,n,n);
```

8.1.2 Note

Matrix3d is complex and rarely used directly. Most 3D effects are better created with individual transform functions.

8.2 Combining with JavaScript for Interactivity

```
<style>  
  .interactive-box {  
    width: 200px;  
    height: 200px;  
    background: blue;  
    perspective: 1000px;  
    transition: transform 0.3s ease;  
  }  
</style>  
  
<div class="interactive-box"></div>  
  
<script>  
  const box = document.querySelector('.interactive-box');  
  let rotX = 0, rotY = 0;  
  
  document.addEventListener('mousemove', (e) => {  
    const x = e.clientX / window.innerWidth;
```

```

    const y = e.clientY / window.innerHeight;

    rotX = (y - 0.5) * 30;
    rotY = (x - 0.5) * 30;

    box.style.transform =
      `rotateX(${rotX}deg) rotateY(${rotY}deg)`;
  });
</script>

```

8.3 Parallax 3D Effect

```

<style>
  .parallax-section {
    height: 100vh;
    perspective: 1000px;
    overflow: hidden;
  }

  .parallax-layer {
    position: fixed;
    width: 100%;
    height: 100%;
    background-size: cover;
  }

  .layer-1 { transform: translateZ(0px); z-index: 1; }
  .layer-2 { transform: translateZ(-100px); z-index: 0; }
  .layer-3 { transform: translateZ(-200px); z-index: -1; }
</style>

<div class="parallax-section">
  <div class="parallax-layer layer-1"
    style="background-image: url('bg1.jpg')"></div>
  <div class="parallax-layer layer-2"
    style="background-image: url('bg2.jpg')"></div>
  <div class="parallax-layer layer-3"
    style="background-image: url('bg3.jpg')"></div>
</div>

```

8.4 Performance Optimization

```

/* Good - GPU accelerated */
.element {
  transform: rotateX(45deg);
  transform-style: preserve-3d;
  will-change: transform;
}

```

```
}

/* Avoid - CPU intensive */
.element {
  left: 10px;
  perspective: 1000px; /* on every element */
  animation: complex-3d 10s ease infinite;
}

/* Better approach */
.container {
  perspective: 1000px;
}

.element {
  transform-style: preserve-3d;
  /* only animate transforms */
  animation: rotate-only 2s linear infinite;
}

@keyframes rotate-only {
  from { transform: rotateX(0); }
  to { transform: rotateX(360deg); }
}
```

Chapter 9

Browser Support & Best Practices

9.1 Browser Support

CSS 3D transforms are supported in modern browsers:

- Chrome 12+
- Firefox 10+
- Safari 4+ (with -webkit- prefix on older versions)
- Edge 12+
- Mobile browsers (most modern versions)

9.1.1 Vendor Prefixes

For older browsers, use prefixes:

```
.element {  
  -webkit-transform: rotateY(45deg);  
  -moz-transform: rotateY(45deg);  
  transform: rotateY(45deg);  
}
```

9.2 Performance Best Practices

1. **Use GPU properties:** Transform and opacity only
2. **Apply perspective once:** On container, not elements
3. **Use will-change wisely:** Only for animated elements
4. **Test on mobile:** 3D performance varies
5. **Limit 3D elements:** Too many can impact performance

9.3 Common Mistakes

9.3.1 Mistake 1: Forgetting perspective

```
/* Wrong - no 3D effect */
.element { transform: rotateY(45deg); }

/* Correct - with perspective */
.container { perspective: 1000px; }
.element { transform: rotateY(45deg); }
```

9.3.2 Mistake 2: Not Using transform-style: preserve-3d

```
/* Wrong - children lose 3D */
.parent { transform: rotateX(45deg); }

/* Correct - preserve 3D space */
.parent {
  transform: rotateX(45deg);
  transform-style: preserve-3d;
}
```

9.3.3 Mistake 3: Incorrect backface-visibility

```
/* Without backface-visibility on card flip */
.card-front, .card-back {
  /* You'll see both sides overlapping */
}

/* With backface-visibility */
.card-front, .card-back {
  backface-visibility: hidden; /* Clean flip */
}
```

9.4 Tips for Better 3D Effects

- **Start simple:** Master rotateY before combining effects
- **Use reasonable perspective:** 800px-1200px works well
- **Test in all browsers:** 3D support varies
- **Combine with transitions:** Makes animations smooth
- **Use backface-visibility:** For flip effects, prevents seeing back

Chapter 10

Quick Reference Guide

10.1 3D Transform Functions

- **rotateX(angle)**: Tilt forward/backward
 - **rotateY(angle)**: Spin left/right
 - **rotateZ(angle)**: Spin in place
 - **rotate3d(x,y,z,angle)**: Custom axis rotation
 - **translateZ(value)**: Move toward/away viewer
 - **translate3d(x,y,z)**: Move in all directions
 - **scale3d(x,y,z)**: Scale in 3D space
-

10.2 3D Properties

- **perspective**: Create 3D space (on parent)
 - **perspective-origin**: Set vanishing point
 - **transform-style**: preserve-3d for child positioning
 - **backface-visibility**: hidden to hide back faces
 - **transform-origin**: Change rotation point
-

10.3 Common Combinations

10.3.1 Card Flip

```
.container { perspective: 1000px; }  
.card { transform: rotateY(180deg); }
```

10.3.2 3D Rotate and Float

```
transform: rotateX(20deg) rotateY(20deg) translateZ(50px);
```

10.3.3 Cube

```
.container { perspective: 1000px; }
.cube { transform-style: preserve-3d; }
.face { backface-visibility: hidden; }
```

10.3.4 Spinning Element

```
animation: spin 2s linear infinite;
@keyframes spin {
  from { transform: rotateX(0) rotateY(0); }
  to { transform: rotateX(360deg) rotateY(360deg); }
}
```

10.4 Do's and Don'ts

Do	Don't
Use perspective on parent	Apply perspective to every element
Use preserve-3d	Forget to use it for nested 3D
Use backface-visibility	Animate position/width instead
Test on mobile	Assume all devices support 3D
Combine transforms wisely	Use matrix3d unnecessarily

10.5 Perspective Value Guide

- **200px**: Extreme 3D (dramatic)
- **500px**: Strong 3D (noticeable)
- **800px**: Moderate 3D (balanced)
- **1000px**: Subtle 3D (natural)
- **1500px**: Very subtle 3D (gentle)

10.6 CSS 3D Transform Checklist

1. Set `perspective` on parent container
2. Use `transform-style: preserve-3d` for nested 3D
3. Apply transform functions to child elements
4. Use `backface-visibility: hidden` for flip effects
5. Combine with `transition` for smooth animations
6. Test in multiple browsers
7. Verify performance on mobile devices
8. Add fallbacks for unsupported browsers